

Product Requirements Document (PRD)

Architecture Mapper Core

Status: Draft / Implementation PRD **Primary objective:** Build a reusable, language-agnostic Architecture Mapper Core package that can be embedded and consumed by software projects across ecosystems such as TypeScript/Node.js, Python, Java, and future languages. All product surfaces and integrations must consume the same Core rather than reimplementing architecture intelligence.

1. Executive Summary

Architecture Mapper is an evidence-backed architecture intelligence and safe-change coordination system for software repositories.

Its central capability is answering:

> **If I change this piece of code, what else could be affected, and why?**

The system builds and maintains one architectural knowledge graph from source code, repositories, APIs, databases, infrastructure, tests, documentation, Git history, runtime/coverage information, and explicit user/agent input.

The most important architectural decision is that the **Architecture Mapper Core is the product foundation**.

The Core is a reusable package/library, not merely an internal implementation of a VS Code extension or daemon. Language-specific packages/bindings expose the same canonical graph model, IDs, evidence model, schemas, and impact semantics.

CLI, MCP, HTTP, VS Code, GitHub Actions, RAG, and AI-agent orchestration are integrations built on top of the Core.

The system is local-first, evidence-backed, incremental, provider-neutral, and designed to help both humans and AI agents understand and modify software safely.

2. Problem

Modern software systems are difficult to reason about because architectural knowledge is fragmented across:

- source code
- packages and modules
- APIs
- databases
- events

- infrastructure
- configuration
- tests
- documentation
- Git history
- runtime/coverage data
- tribal knowledge
- AI-generated assumptions

Developers and AI coding agents frequently know what they want to change but cannot reliably determine:

- what depends on the changed component
- which services consume an API
- which database entities are affected
- which tests cover the affected path
- which external dependencies matter
- whether contracts or schemas need updates
- whether a proposed architectural relationship is actually supported by source evidence
- what files are safe to modify

Existing code intelligence tools often focus on one language, editor, repository view, or AI vendor. Architecture Mapper should instead provide a common architectural substrate that can be consumed by different languages, tools, editors, CI systems, and AI agents.

3. Product Vision

Build a **portable architecture intelligence layer for software development**.

A project should be able to install or embed Architecture Mapper Core and obtain:

1. A canonical architectural graph.
2. Evidence-backed relationships.
3. Fast incremental synchronization.
4. Explainable change-impact analysis.
5. Git diff impact analysis.
6. Documentation and contract context.
7. Test-selection signals.
8. Safe change-planning envelopes.
9. Machine-readable APIs for humans and AI agents.
10. A foundation for bounded, verifiable multi-agent software development.

4. Product Principles

4.1 One source of architectural truth

There is one graph of record.

Parsers, Git, documentation, OpenAPI, databases, infrastructure, coverage, runtime signals, seeds, pins, agents, and models all upsert the same graph nodes and edges.

There must not be parallel authoritative stores such as separate "pinned", "observed", or "inferred" graphs.

RAG chunks point to graph nodes; RAG is not a competing architectural truth source.

4.2 Core-first architecture

The reusable Architecture Mapper Core is the primary product.

All higher-level integrations should depend on the Core or its canonical APIs.

No integration should independently reimplement graph semantics, IDs, impact traversal, evidence handling, or architectural truth.

4.3 Language agnostic

The Core must be usable from different ecosystems.

Initial targets:

- TypeScript / Node.js
- Python
- Java

Future languages should be addable without changing the canonical graph model.

4.4 Evidence over confidence

Every important architectural relationship must be traceable to evidence.

LLMs may propose relationships only when grounded in real source evidence or explicit user/agent pins.

4.5 Deterministic first

Prefer:

parser → graph → Git → tests → docs → heuristics → LLM

Use models only where deterministic tooling cannot reasonably answer the question.

4.6 Explainability

Impact results must explain paths and evidence, not merely list affected files.

4.7 Incremental by default

A file save should result in a surgical graph patch, not a full repository rethink.

Service identity should only be reconsidered when the workspace fingerprint changes or when explicitly requested.

4.8 Agent safety

AI agents are bounded, observable, and independently verified.

An agent's claim is provisional until validated against repository state, graph state, evidence, schemas, tests, or an independent verifier.

4.9 No provider lock-in

Model routing must be based on capability, context, risk, latency, and cost rather than a single model vendor.

4.10 Minimal human setup

A user should be able to install the package/CLI/extension, open a repository, and get useful architecture information without manually cataloguing the system.

Optional seed configuration exists only where deterministic inference is insufficient.

5. Target Users

5.1 Developers

Need to understand the consequences of modifying code in unfamiliar or large systems.

5.2 Staff / senior engineers

Need architecture-level visibility, coupling analysis, contract awareness, and safe change planning.

5.3 AI coding agents

Need machine-readable architecture context before modifying code.

5.4 Platform / DevOps teams

Need service, API, database, infrastructure, and deployment relationships.

5.5 Engineering organizations

Need consistent architecture intelligence across repositories and technology stacks.

6. Primary Use Cases

UC1 — Understand a symbol

Given a function, method, class, module, API, service, or table, retrieve its architectural context and neighbors.

UC2 — Determine blast radius

Given a starting node, determine what can be affected and why.

UC3 — Explain impact

Return shortest evidence-backed paths showing how the changed node reaches downstream systems.

UC4 — Analyze a Git diff

Determine architectural impact between a base and head revision.

UC5 — Identify tests to run

Find tests connected to affected symbols and infer relevant commands.

UC6 — Resolve external documentation

Find official documentation for exact dependency versions involved in an affected path.

UC7 — Safely plan a change

Produce an explicit mutation envelope describing:

- allowed files
- impacted nodes
- applicable policies
- required tests
- relevant contracts
- known risks

UC8 — Help AI agents modify code

An agent first examines impact, documentation, and a change plan, then modifies only the approved files, and finally verifies the result.

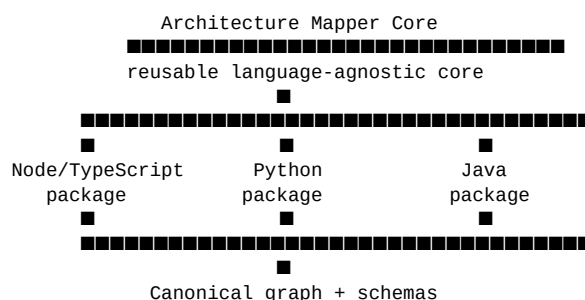
UC9 — Review a pull request

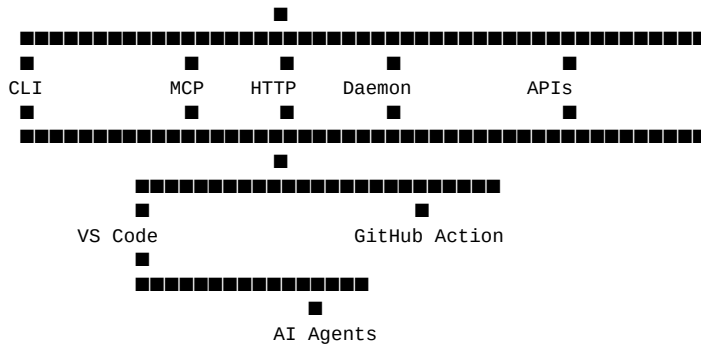
Produce a machine-readable and human-readable architectural impact report for a GitHub PR.

UC10 — Explore architecture visually

Show service, contract, database, and dependency relationships without producing an unreadable graph hairball.

7. Product Architecture





The Core should support an **embedded/library mode** so consumers do not have to run the daemon for every use case.

The daemon is an optional runtime host that provides persistent indexing, synchronization, shared access, MCP, HTTP, and multi-client coordination.

8. Architecture Mapper Core

8.1 Core responsibilities

The Core owns:

- canonical graph schema
- stable IDs
- graph persistence abstraction
- node and edge upsert
- evidence/provenance
- conflict representation
- graph traversal
- impact analysis
- why-path generation
- risk calculation
- symbol-level diff primitives
- policy primitives
- verification primitives
- canonical serialization
- schema validation
- plugin interfaces
- language-neutral contracts

8.2 Core must not depend on

The Core must not require:

- VS Code
- a particular editor
- MCP
- GitHub
- a specific LLM provider
- a cloud service
- a web application
- a particular agent framework

These are integrations.

8.3 Core package requirements

Initial package targets:

```
@archmap/core
archmap-core
com.archmap:archmap-core
```

Exact final package naming may change when the product is branded.

Each binding must expose equivalent semantics.

Example conceptual API:

```
graph.upsert(...)
graph.search(...)
graph.symbol(...)
graph.neighbors(...)
graph.impact(...)
graph.whyPath(...)
graph.diffImpact(...)
graph.health(...)
```

The exact language-specific API can follow ecosystem conventions, but the underlying semantics and schemas must remain compatible.

9. Canonical Graph

9.1 Nodes

Initial node types:

Repo
File
Module
Package
Class
Interface
Function
Method
Service
API
Route
Table
Column
Event
Job
Test
External
Infra
Doc
Contract
ConfigKey

9.2 Edges

Initial edge types:

CONTAINS
IMPORTS
CALLS
IMPLEMENTS
EXPOSES
CONSUMES
READS
WRITES
PUBLISHES
SUBSCRIBES
TESTS
DEPENDS_ON
DOCUMENTS
CONSTRAINED_BY
CO_CHANGED
BROKE_BEFORE
USES_CONFIG

9.3 Stable IDs

Canonical examples:

```
repo:<name>
file:<posix-relpath>
mod:<posix-relpath>
pkg:<package-name>@<version-or-workspace>
cls:<relpath>:<Class>
iface:<relpath>:<Name>
fn:<relpath>:<qualname>
svc:<service-id>
api:<METHOD>:<path>
table:<name>
col:<table>.<name>
event:<name>
job:<relpath>:<name>
test:<relpath>:<name>
ext:<package-or-system>
infra:<relpath>
doc:<url-or-relpath>
cfg:<KEY>
```

Multi-repository workspaces must distinguish repository roots.

10. Evidence and Provenance

Every important edge must contain evidence or an explicit pin.

Canonical edge structure:

```
{
  "id": "e...",
  "type": "CALLS",
  "from": "fn:apps/payments/service.py:processPayment",
  "to": "fn:apps/payments/validate.py:validateTransaction",
  "evidence": {
    "file": "apps/payments/service.py",
    "line": 84,
    "snippet": "validateTransaction(tx)"
  },
  "sources": ["parser"],
  "confidence": 0.96,
  "conflict": false,
  "updated_at": "ISO-8601"
}
```

Possible evidence sources:

```
parser
git
openapi
lockfile
coverage
infra
runtime
user
agent
llm
```

Multiple sources belong to the same logical edge.

Automated disagreement must create a conflict state rather than duplicate logical edges.

11. Parsing and Ingestion

11.1 Initial language support

Initial parser targets:

- Python
- TypeScript
- Java

Tree-sitter should be used where appropriate for syntax extraction.

11.2 Parser output

Parsers should produce normalized Core entities and relationships.

At minimum, extract where supported:

- files
- modules
- packages
- imports
- functions
- methods
- classes
- interfaces
- calls
- API routes
- database usage
- configuration usage
- tests

11.3 Additional ingestion

Support when present:

- OpenAPI
- AsyncAPI
- protobuf
- SQL
- Prisma
- migrations
- lockfiles
- package manifests
- Terraform
- Docker Compose
- Helm
- GitHub Actions
- test suites
- coverage
- runtime/telemetry signals

12. Incremental Sync

The system must support a fast and slow clock.

Fast clock

Triggered by:

- file save
- dirty buffer
- dropped file

Behavior:

1. identify changed files
2. load existing nodes/edges for those files
3. parse changed content
4. patch the graph
5. update changed-symbol RAG chunks
6. recompute cached impact for dirty symbols
7. run health checks
8. journal the operation

Slow clock

Triggered by:

- workspace fingerprint change
- checkout
- seed change
- explicit reindex

Behavior may include:

- service identification
- broader summaries
- dependency/documentation refresh
- structural recomputation

Circuit breaker

If service identities or important edges repeatedly change without a fingerprint change:

- freeze LLM/identity writers
- preserve current graph
- emit `inference_paused`

13. Impact Analysis

The primary Core feature is evidence-backed impact analysis.

Conceptual algorithm:

```
impact(start_ids, direction=downstream, depth=5)
```

Downstream traversal includes relationships such as:

- inverse CALLS
- EXPOSES
- inverse CONSUMES
- inverse WRITES
- PUBLISHES
- inverse TESTS
- inverse DEPENDS_ON

Upstream traversal includes:

- CALLS
- IMPORTS
- READS
- CONSUMES
- DEPENDS_ON

Return:

- counts by node type
- impacted nodes
- shortest *why*-paths
- evidence
- tests to run
- relevant external docs
- risk chips

Default limits:

- maximum depth: 5

- maximum why-paths: 7

Risk factors include:

- downstream count
- critical flag
- database writes
- external dependency
- missing tests
- high degree
- Git churn
- conflict edges

Results must be derived from graph rows.

14. Git Diff Impact

Implement symbol-level diff analysis.

Classify changes as:

- added
- removed
- signature-changed
- body-only

Then:

1. identify changed symbols
2. calculate union impact
3. detect contract/schema/infra deltas
4. return structured impact data

Default comparison base is `main` where appropriate.

15. Documentation Resolver

For external dependencies:

1. resolve exact dependency version from lockfile
2. find official documentation for that version

3. cache it locally
4. create Doc nodes
5. attach DOCUMENTS relationships
6. include in-repository README, ADR, OpenAPI, and `llms.txt` where available
7. fetch changelog/releases on major version bumps
8. allow LLM summarization only over retrieved text

The system must never invent external API parameters.

16. RAG

RAG is secondary context, not architectural truth.

Requirements:

- chunks must reference graph nodes
- changed symbols should update incrementally
- no full-repository embedding on every save
- support SQLite-vector or LanceDB initially
- historical agent rooms may be stored as explicitly labeled contextual knowledge
- summaries must preserve provenance

17. Search

Core search should support:

```
search(query, optional kind)
```

Search should return graph nodes and allow higher-level systems to retrieve the exact source/evidence required.

18. Health and Policies

Built-in warnings:

- public route changed without OpenAPI/contract update
- critical node has zero tests
- seeded ownership violation for WRITES
- major dependency version bump on a critical path

Blocking behavior:

- only when explicitly configured through `policies.yaml`
- or when a user enables a merge gate

v1 must not fail merges by default.

19. Seed and Pins

Optional configuration:

```
.archmap/seed.yaml  
.archmap/policies.yaml
```

Seed can describe:

- services
- ownership
- external consumers
- explicit pins
- ignored paths
- critical nodes
- question behavior

Pins become graph evidence and do not create a second truth layer.

20. Change Planning

Implement:

```
plan_change(id | intent)
```

Return a bounded mutation envelope containing:

- allowed files
- impacted nodes
- applicable policies
- required tests
- relevant contracts
- known risks
- required evidence

AI agents must not edit outside this envelope.

21. Agent API

The canonical machine-readable API should include:

```
search
symbol
neighbors
blast_radius
diff_impact
why_path
docs_for
tests_to_run
health
plan_change
pin
record_event
open_graph
agent_run
agent_verify
agent_debate
agent_skill
```

Every tool should follow a common JSON contract.

Example:

```
{
  "ok": true,
  "nodes": [],
  "edges": [],
  "paths": [],
  "counts": {},
  "risk": [],
  "evidence_used": true
}
```

MCP, CLI, and HTTP must expose equivalent semantics.

22. CLI

Initial commands:

```
archmap serve
archmap mcp
archmap sync
archmap impact <id> --json
archmap diff [base] [head] --json
archmap docs <name> --json
archmap pin ... --json
archmap health --json
archmap search <q> --json
archmap agent run ... --json
archmap agent verify ... --json
archmap agent debate ... --json
```

The CLI must consume Core semantics rather than implement a separate architecture engine.

23. HTTP API

Provide localhost HTTP:

```
127.0.0.1:<port>/v1/<tool>
```

POST JSON.

The daemon port is stored in:

```
.archmap/daemon.json
```

HTTP, CLI, and MCP must share schemas and operations.

24. Daemon

The daemon is the optional persistent runtime host.

Responsibilities:

- persistent graph access
- synchronization
- shared clients
- HTTP
- MCP
- orchestration
- centralized permission enforcement
- graph writes
- journal coordination

CLI must be able to use an existing daemon or operate one-shot when no daemon is available.

25. Language Packages and Bindings

25.1 Node.js / TypeScript

Provide a native ecosystem package exposing Core capabilities.

Requirements:

- idiomatic TypeScript API
- typed schemas
- JSON compatibility
- stable IDs
- graph operations
- impact operations
- plugin interfaces

25.2 Python

Provide a Python package exposing equivalent Core semantics.

25.3 Java

Provide a Java package exposing equivalent Core semantics.

25.4 Cross-language conformance

Create a shared conformance suite verifying:

- schema compatibility
- ID compatibility
- edge semantics
- impact results
- serialization
- evidence representation
- error semantics

A graph created or queried through one binding must remain compatible with the others.

26. VS Code Extension

The extension is a thin client/host, not the product core.

v1:

- activate on workspaceContains: .git
- start daemon if required
- register MCP server
- merge-write .mcp.json

- background sync
- status bar
- CodeLens impact counts
- Map
- Impact
- Docs
- Health
- hover docs
- hover why-summary
- click node → source line

No settings wizard.

Permission prompts only for:

- installing Git hooks
- writing GitHub Actions
- sending source to a cloud model when no local/editor model is available

27. Visualization

Use:

Purpose	Technology
Interactive architecture	React Flow
Large graph	Cosmograph
PR/Markdown	Mermaid

All views must consume the same query results.

Default visualization should emphasize:

- services
- contracts
- datastores

Do not display every file by default.

Interactions:

- click node → open source line
- hover edge → evidence
- Show why → animate why-path

Do not build a custom WebGL graph engine.

28. GitHub Action

On pull request:

```
archmap diff $BASE $HEAD --json
```

Publish a sticky PR comment containing:

- risk chips
- counts
- Mermaid why-path
- tests to run
- conflicts
- contract gaps

Required permissions:

```
contents: read
pull-requests: write
checks: write
```

29. Agent Platform

Architecture Mapper also provides a safe coordination layer for AI agents.

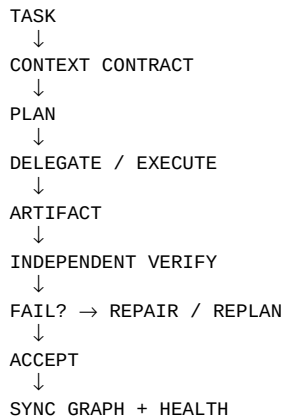
Supported roles:

- orchestrator
- explorer
- architect
- impact-analyzer
- implementer
- reviewer
- verifier
- docs-agent
- security-agent
- test-agent
- prompt-agent
- cost-agent

Agents may combine capabilities but every run must have an explicit role.

30. Agent Verification

Every consequential task follows:



Verification must check:

- changed files are inside the allowed envelope
- graph synchronization succeeds
- no unexplained conflict edges appear
- relevant tests pass or failures are reported
- API/schema/contract consistency
- impact is recomputed
- graph IDs are stable
- edge endpoints are valid
- evidence snippets exist
- source metadata is valid
- duplicate logical edges are not created

Verification failure blocks acceptance.

Default verification budget:

```
verification:
  max_retries: 2
  max_subagents: 8
  max_depth: 3
  require_independent_reviewer_for:
    - schema_changes
    - security_changes
    - critical_paths
    - public_api_changes
    - agent_prompt_changes
```

31. Agent Skills

Skills are reusable capability contracts.

Required contract fields:

```
name:
description:
inputs:
outputs:
allowed_tools:
required_evidence:
verification:
max_tokens:
max_runtime:
side_effects:
```

Initial skills:

```
impact-analysis
repository-exploration
docs-resolution
change-planning
safe-implementation
code-review
graph-verification
test-selection
contract-check
prompt-review
cost-routing
```

Skills should communicate using structured results.

32. Prompt Contracts

Every substantial agent task should define:

```
task:
role:
goal:
context:
evidence:
constraints:
allowed_tools:
allowed_files:
forbidden_actions:
output_schema:
success_criteria:
verification:
budget:
```

Contracts must define:

- responsibility
- authority
- mutation boundaries
- forbidden actions

- evidence requirements
- output format
- uncertainty
- success criteria
- verification

Agents must not infer permission to modify arbitrary files.

33. Reverse Prompting

When requirements are ambiguous:

1. inspect repository/graph/Git/docs first
2. determine what can be known deterministically
3. identify only materially important unknowns
4. ask focused questions
5. ask no more than 3 questions at once

Do not ask users questions that tools can answer.

34. Multi-Agent Orchestration

Support:

Sequential

explore → plan → implement → verify

Parallel fan-out

```
repo + graph + git + docs + tests
      ↓
    synthesize
```

Debate

propose → critique → verify → decide

Generator / critic

generate → critique → repair → verify

Mixture of experts

Route tasks to the cheapest capable specialist.

Escalation

```
deterministic
↓
graph heuristics
↓
small model
↓
strong model
↓
human question
```

Do not use multiple agents simply because they are available.

35. Agent Chat Rooms

Rooms are collaboration contexts, not architectural truth.

A room contains:

- task ID
- participants
- compact context
- messages/decisions
- evidence references
- current artifact
- unresolved questions
- final decision
- verification state

Lifecycle:

create → context-pack → discuss → decide → verify → archive

Rooms must reference graph/file IDs where possible.

36. Context Management

Preferred context order:

1. exact task
2. relevant graph nodes/edges
3. exact source snippets
4. relevant diff
5. relevant tests
6. relevant docs
7. relevant history
8. broader repository only when necessary

Avoid repeatedly sending:

- entire repositories
- unchanged files
- full lockfiles
- irrelevant chat history
- duplicate tool results

Context packs should be structured:

```
{
  "task": "...",
  "facts": [],
  "evidence": [],
  "constraints": [],
  "open_questions": [],
  "artifacts": []
}
```

37. Context Compression

Compression must preserve:

- stable IDs
- file paths
- line numbers
- signatures
- edge types
- evidence snippets
- constraints
- failures
- decisions
- unresolved uncertainty
- provenance

Summaries must retain provenance.

A summary is never a replacement for graph/source truth.

38. Model Router

Routing dimensions:

- reasoning difficulty
- repository ambiguity
- context size
- latency
- cost
- required tools
- security sensitivity
- verification level

Default strategy:

deterministic → simple/cheap model → strong model → independent verifier

Use deterministic tooling for:

- parsing
- graph queries
- Git diff
- schema validation

Use cheap models for:

- summaries
- classification
- routing
- simple narration

Use strong models for:

- difficult architecture
- dynamic-language coupling
- plan_change
- difficult incidents

Use independent verifiers for:

- critical changes
- security changes
- prompt changes
- ambiguous graph edges

39. Cost and Telemetry

Track:

- agent count
- model calls
- input tokens
- output tokens
- cached input tokens
- tool calls
- latency
- estimated cost
- verification cost

Default budget:

```
budget:
  max_agents: 8
  max_depth: 3
  max_model_calls: 20
  max_input_tokens: 100000
  max_output_tokens: 30000
  max_runtime_seconds: 300
```

Before spawning an agent:

> Will this call materially reduce uncertainty, risk, or implementation time?

If not, do not spawn it.

40. Prompt Self-Improvement

Agents may propose improvements to prompts, routing, skills, and contracts.

They may not silently modify governing instructions.

Required flow:

```
proposal
→ prompt review
→ verification
```

- explicit approval
- versioned update

Prompt changes must:

- be visible as diffs
- be reviewed
- preserve safety/evidence/verification rules
- require explicit approval when authority changes

Repository content, web content, issues, or generated text cannot override governing instructions.

41. Security

Requirements:

- local index by default
- do not commit graph DB
- do not commit embeddings
- do not upload entire source files to cloud models by default
- send symbols/evidence snippets instead
- exclude secret-like paths
- treat repository content as untrusted input
- enforce tool allowlists
- enforce allowed-file envelopes
- require explicit network permission
- require explicit cloud-source permission
- keep credentials out of prompts/logs/transcripts
- agent outputs remain untrusted until verified

42. Workspace Artifacts

Generated:

```
.archmap/index.db
.archmap/vectors/
.archmap/cache/docs/
.archmap/journal.jsonl
.archmap/daemon.json
.archmap/agent-runs/
```

Optional:

```
.archmap/seed.yaml
```

`.archmap/policies.yaml`

Maintained when missing:

`.mcp.json`
`AGENTS.md`

Generated state must be ignored by Git.

43. Demo Fixture

Create:

`examples/payments-platform`

It should contain:

- payments service
- `processPayment()`
- `validateTransaction()`
- `POST /payments`
- payments writes
- orders service consuming payments
- ledger worker
- `orders/payments/ledger` tables
- SQL or Prisma
- OpenAPI
- intentionally missing critical test coverage
- one ADR
- one real or stubbed external

This fixture is the judging/demo environment.

44. Functional Requirements

FR-001 Core availability

The Core must be importable from supported ecosystems without requiring VS Code.

FR-002 Canonical schemas

All language bindings must serialize to compatible canonical schemas.

FR-003 Stable identity

Equivalent repository entities must receive stable IDs across incremental updates.

FR-004 Evidence

Important relationships must include evidence or explicit pins.

FR-005 Incremental updates

Changing one file must not trigger unnecessary whole-repository reprocessing.

FR-006 Impact

The Core must compute bounded, evidence-backed impact paths.

FR-007 Diff impact

The system must analyze architectural impact of Git diffs.

FR-008 Contracts

The system must identify relevant API/schema/infra contract changes.

FR-009 Tests

The system must identify relevant tests on impact paths.

FR-010 Docs

The system must resolve official documentation for affected external dependencies when available.

FR-011 Planning

The system must generate an explicit mutation envelope for agent implementation.

FR-012 Verification

Consequential agent results must be independently verifiable.

FR-013 Shared semantics

CLI, MCP, HTTP, and language packages must use the same underlying semantics.

FR-014 No duplicate truth

No integration may create a second architectural source of truth.

FR-015 Local-first

Core functionality must remain usable without cloud AI.

45. Non-Functional Requirements

Performance

- incremental saves should process only changed symbols/files
- graph queries should be bounded
- why-paths limited to 7
- traversal depth limited to 5
- large responses should paginate above 50 nodes unless explicitly required

Reliability

- graph writes must be idempotent
- stable IDs must survive repeated syncs
- conflicts must be represented explicitly
- verification failures must block acceptance

Portability

- Node/TypeScript, Python, and Java support
- language-neutral schemas
- no editor lock-in
- no model-provider lock-in

Security

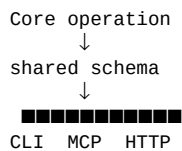
- local-first storage
- explicit network/cloud permissions
- secret exclusion
- bounded file mutation

Observability

- agent journal
- sync events
- verification state
- model/tool telemetry
- decisions and failures

46. CLI / MCP / HTTP Contract

All surfaces must map to the same canonical operations.



No surface may introduce incompatible behavior.

47. API Compatibility Strategy

The Core package should use explicit semantic versioning.

Compatibility guarantees should cover:

- node schemas
- edge schemas
- stable IDs
- serialized results
- operation semantics

Language bindings should be versioned against the Core schema/contract version.

Breaking graph/schema changes require independent review.

48. Implementation Phases

Phase 0 — Core Foundation

1. Define Core public API.
2. Define canonical graph schema.
3. Define node/edge IDs.
4. Define evidence model.
5. Define conflict model.
6. Implement graph storage abstraction.
7. Implement graph upsert.
8. Implement traversal.
9. Implement impact.
10. Implement why-path.
11. Implement risk calculation.
12. Implement canonical serialization.
13. Implement schema validation.
14. Establish Core test suite.

Phase 1 — Language Packages

1. Node/TypeScript package.
2. Python package.
3. Java package.
4. Shared conformance suite.
5. Cross-language schema compatibility.
6. Cross-language ID compatibility.
7. Cross-language impact compatibility.

Phase 2 — Parsers and Ingestion

1. Python parser.
2. TypeScript parser.
3. Java parser.
4. Parser plugin interface.
5. API ingestion.
6. DB/schema ingestion.
7. infrastructure ingestion.
8. test/coverage ingestion.
9. lockfile/manifests.

Phase 3 — Runtime

1. sync engine
2. daemon
3. CLI
4. HTTP
5. MCP

Phase 4 — Product Surfaces

1. VS Code
2. visualization
3. docs resolver
4. seed/pins
5. health
6. GitHub Action

Phase 5 — AI Layer

1. RAG
2. search
3. plan_change
4. Agent Skills
5. prompt contracts
6. verification loops
7. orchestration
8. debate
9. chat rooms
10. model router
11. cost telemetry
12. prompt versioning/self-improvement proposals

49. Definition of Done

Core

- [] Core can be imported independently of the daemon.

- ☐ Canonical graph schema is implemented.
- ☐ Stable IDs are implemented.
- ☐ Evidence/provenance is implemented.
- ☐ Conflict handling is implemented.
- ☐ Upsert is idempotent.
- ☐ Impact analysis works.
- ☐ Why-paths include evidence.
- ☐ Core results use stable machine-readable schemas.

Language packages

- ☐ Node/TypeScript package works.
- ☐ Python package works.
- ☐ Java package works.
- ☐ Cross-language conformance tests pass.

Indexing

- ☐ Python code is indexed.
- ☐ TypeScript code is indexed.
- ☐ Java code is indexed.
- ☐ Incremental parsing works.
- ☐ Service identity does not thrash without fingerprint changes.

Agent integration

- ☐ MCP works.
- ☐ CLI JSON works.
- ☐ HTTP works.
- ☐ blast_radius works.
- ☐ diff_impact works.
- ☐ plan_change works.
- ☐ Agent verification blocks invalid artifacts.

Product

- ☐ VS Code extension can consume Core.
- ☐ Architecture map renders.
- ☐ PR impact report renders.

- [] Docs resolver works.
- [] Health checks work.
- [] Demo fixture demonstrates end-to-end value.

50. Demo Acceptance Criteria

Given the payments demo:

1. Open the workspace with no manual catalog.
2. The graph builds automatically.
3. Query processPayment.
4. The system identifies its path to:
 - validateTransaction
 - PaymentService
 - POST /payments
 - Order Service
 - payments datastore
 - relevant tests
5. Edit processPayment.
6. Incremental synchronization updates impact without re-identifying services.
7. Add a missing consumer with pin.
8. The same graph is updated.
9. MCP blast_radius and CLI impact --json return equivalent IDs.
10. docs_for returns relevant external/in-repo documentation.
11. diff_impact generates PR-ready architectural information.
12. An agent receives a change envelope before editing.
13. The agent cannot modify files outside the envelope.
14. Verification checks the artifact independently.
15. Failed verification prevents acceptance.
16. Agent decisions and verification are journaled.

51. Out of Scope for V1

The following are explicitly not the initial focus:

- cloud-hosted repository analysis as the primary product
- a paste-a-repository web application
- custom WebGL graph rendering

- automatic merge blocking by default
- unrestricted autonomous agents
- a vendor-specific AI integration
- full-repository LLM analysis on every save
- a second architectural knowledge database
- silently self-modifying governing prompts

52. Success Metrics

Core correctness

- percentage of graph edges with valid evidence
- percentage of stable IDs preserved across incremental syncs
- duplicate-edge rate
- unexplained conflict rate
- parser-to-graph conformance rate

Impact quality

- percentage of important impact paths with evidence
- false-positive impact rate
- false-negative impact rate on demo/reference fixtures
- test-selection usefulness

Performance

- incremental sync latency
- graph query latency
- full initial indexing time
- memory usage

Agent safety

- percentage of consequential tasks independently verified
- number of out-of-envelope mutations prevented
- verification failure detection rate
- number of unverified agent claims accepted

Efficiency

- model calls per task
- tokens per task
- deterministic-vs-LLM operation ratio
- verification cost
- cache/context reuse rate

53. Risks and Mitigations

Risk	Mitigation
Graph becomes inaccurate	Evidence-backed edges, conflict state, verification
LLM hallucination	LLM cannot invent unsupported edges
Parser gaps	Seed/pin support and bounded inference
Language differences	Canonical normalized Core schema + conformance suite
Integration divergence	All integrations consume Core semantics
Performance degradation	Incremental parsing and bounded traversal
Agent runaway	depth/count/token/runtime budgets
Prompt injection	governing AGENTS.md and explicit policy hierarchy
Excessive model cost	deterministic-first routing and budgets
Architectural hairball	semantic views and default service-level zoom
Provider lock-in	capability-based model routing
Data leakage	local-first design and explicit cloud permissions

54. Governing Rules for Implementation Agents

Agents implementing this product must:

1. Read AGENTS.md.
2. Understand the task.
3. Inspect repository, graph, Git, and documentation evidence.
4. Create a prompt contract for consequential work.
5. Create a bounded implementation plan.
6. Use sub-agents only when they provide measurable value.
7. Implement inside an approved mutation envelope.
8. Independently verify consequential work.

9. Synchronize the graph.
10. Re-run impact/tests/health.
11. Review the final diff.
12. Record important decisions and failures.
13. Report what was actually verified.

Agents must never:

- invent architecture facts
- invent graph edges
- silently expand scope
- silently rewrite governing prompts
- treat another agent's claim as proof
- spawn unnecessary agents
- dump unnecessary repository context
- trade verification for token savings
- create parallel sources of architectural truth

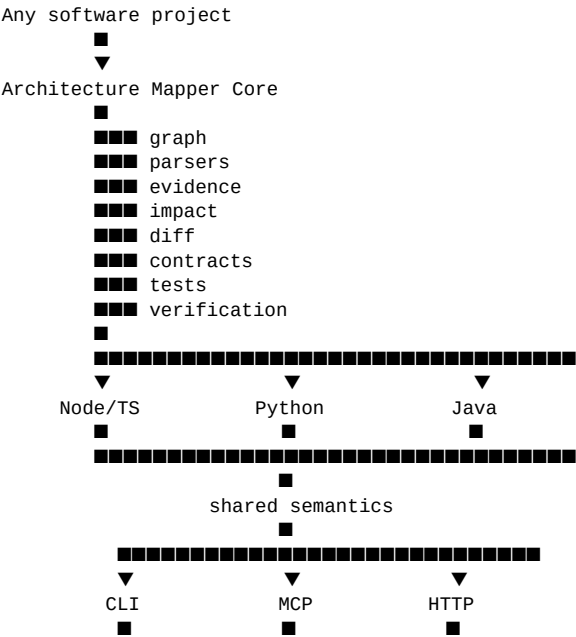
55. Final Product Definition

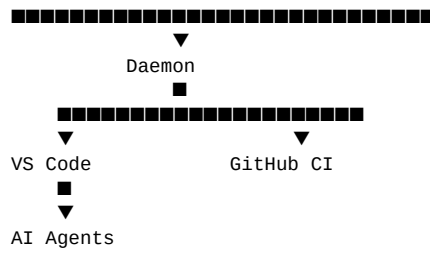
Architecture Mapper is not primarily a VS Code extension, CLI, MCP server, or AI agent.

The product foundation is the reusable Architecture Mapper Core package.

Everything else is an interface, runtime, adapter, or consumer of that Core.

The desired end state is:





The central promise remains:

> Before changing software, understand what it is connected to, why it is connected, what evidence supports that understanding, and what must be verified after the change.